

Relatório: Trabalho 3 - Sistema de Matchmaking

Estrutura de Dados

Alunos: Joel Gil Medeiros de Oliveira | Osmar Caio da Costa Silva

1 Algoritmos de Ordenação

A ordenação dos jogadores foi feita em ordem crescente de *score*. Em caso de empate, o critério de desempate utilizado foi o menor *timestamp*. Essa regra foi centralizada em uma função auxiliar de comparação, usada tanto no *insertion sort* quanto no *merge sort*.

No *insertion sort*, o array é percorrido a partir da segunda posição. Cada jogador é comparado com os anteriores e inserido na posição correta, deslocando para a direita os elementos que devem ficar depois dele. Esse algoritmo é simples, mas possui custo $O(n^2)$ no pior caso.

No *merge sort*, o array é dividido em duas metades até chegar a intervalos de tamanho unitário. Depois, as partes são intercaladas em ordem, com auxílio de arrays temporários. Como há aproximadamente $\log n$ níveis de divisão e cada nível percorre os elementos uma vez, o custo é $O(n \log n)$.

2 Recursão no Merge Sort

A recursão foi usada no método auxiliar do *merge sort*. A cada chamada, o intervalo entre `left` e `right` é dividido em duas partes: a metade esquerda e a metade direita. Cada metade é ordenada recursivamente e, ao final, as duas partes são unidas pelo método de *merge*.

A condição de parada ocorre quando o intervalo possui apenas um elemento, pois um único jogador já está ordenado. Assim, a ordenação completa é construída pela combinação de vários trechos menores já ordenados.

3 Formação de Grupos

O método `formGroup` assume que os jogadores já estão ordenados por *score*. Isso facilita a busca, pois jogadores com níveis próximos tendem a ficar em posições próximas no array.

O sistema considera apenas grupos consecutivos de tamanho `groupSize`. Para verificar se um grupo é válido, basta comparar o *score* do último jogador do trecho com o *score* do primeiro. Se a diferença for menor ou igual a `delta`, o grupo é aceito.

Quando um grupo válido é encontrado, os jogadores são copiados para um novo array alocado dinamicamente e removidos da fila. A remoção é feita deslocando os elementos posteriores para a esquerda. Caso nenhum grupo válido seja encontrado, o método retorna `nullptr` e mantém a fila inalterada.

4 Custo Computacional

Seja n a quantidade de jogadores armazenados. A inserção custa $O(1)$, pois o jogador é colocado diretamente no final do array. A remoção por ID custa $O(n)$ no pior caso, pois pode exigir busca linear e deslocamento dos elementos posteriores.

A formação de grupo custa $O(n)$, já que percorre janelas consecutivas do array e, caso encontre um grupo, pode deslocar elementos após a remoção. O método `getWaitingPlayers` custa $O(n)$, pois cria uma cópia dos jogadores armazenados. A impressão da fila também custa $O(n)$.

Operação	Custo no pior caso
Inserção	$O(1)$
Remoção por ID	$O(n)$
Insertion Sort	$O(n^2)$
Merge Sort	$O(n \log n)$
Formação de grupo	$O(n)$
Recuperação dos jogadores	$O(n)$
Impressão da fila	$O(n)$

Tabela 1: Custo computacional das principais operações.

5 Comparação Experimental

Para comparar os algoritmos, foram criados jogadores com identificadores sequenciais, nome genérico, *timestamps* crescentes e *scores* variados. Os dois algoritmos receberam entradas equivalentes, permitindo uma comparação justa.

Quantidade de jogadores	Insertion Sort (ms)	Merge Sort (ms)
500	2.09	0.36
1.000	8.48	0.78
2.000	35.84	1.93
4.000	139.90	3.94
8.000	555.73	8.39
16.000	2251.19	18.02

Tabela 2: Comparação experimental entre os algoritmos de ordenação.

Os resultados estão de acordo com as complexidades esperadas. No *insertion sort*, quando a entrada dobra, o tempo cresce aproximadamente quatro vezes: de 500 para 1.000 jogadores, o tempo passa de 2,09 ms para 8,48 ms; de 1.000 para 2.000, passa para 35,84

ms; e de 8.000 para 16.000, passa de 555,73 ms para 2251,19 ms. Esse comportamento é compatível com $O(n^2)$, pois duplicar n tende a multiplicar o custo por cerca de $2^2 = 4$.

No *merge sort*, o crescimento é bem menor. Ao duplicar a entrada, o tempo aumenta um pouco mais que o dobro, pois o custo esperado é $O(n \log n)$. Por exemplo, de 8.000 para 16.000 jogadores, o tempo passa de 8,39 ms para 18,02 ms.